

SINGAPORE-MIT ALLIANCE

HIGH PERFORMANCE COMPUTATION FOR ENGINEERED SYSTEMS CME5232: Cluster and Grid Computing Technologies for Scientific Computing COMPUTATIONAL LAB No. 2 13th July, 2007

GETTING STARTED ON PARALLEL PROGRAMMING USING MPI AND ESTIMATING PARALLEL PERFORMANCE METRICS

Objectives

To experiment with MPI library functions for parallelizing sample codes, use MPI functions for point-to-point and collective communications and estimate parallel performance metrics.

Background Knowledge:

C/C++ languages, Basic MPI Functions and some codes from Computational Lab 1.

Parallelization Using MPI

There are a number of parallel software environments that allow users to use networked computers as a unified computing resource, e.g. PVM (Parallel Virtual Machine), MPI (Message Passing Interface) and so on. MPI is a set of specifications that has become a de-facto standard in message passing protocols. MPI has been implemented through PVM, P4, CHIMP MPICH and LAM. MPI is designed to be language independent (it can work with C and Fortran) and of computer architecture. It is designed to resolve all heterogeneities encountered in distributed computing in a transparent manner. MPI is flexible and offers users a variety of binding functions to write efficient parallel codes. There are many MPI functions that implement many aspects of parallelization but only about a dozen or so are really needed frequently. The focus of this lab is on exploiting these bare minimum MPI functions to help you get started on writing parallel programs using MPI. MPICH, an implementation of MPI, is available on the SMA Hydra Linux Cluster for all your parallelization work.

Exercises (Due date: 20th July 2007)

Please submit a short formal report recording your observations based on the assigned exercises and programming assignment in a week's time. The report should be short, complete and clearly written to show what you have done and what you have observed and interpreted from these exercises. All equations, symbols etc. if any must be explained clearly.

1. **Play Ping-Pong Using 2 Processors on the SMA Hydra Cluster**

Write a program using simple send and receive MPI functions in which 2 processors are sending their process ranks to each other and at the end they both print the values of what they sent as well as what they received. A sample skeleton source code **pingpong.c** or **pingpong.cpp** is provided for your reference.

2. **Estimate bandwidth and message latency on the SMA Hydra Cluster**

Write a short program to estimate the message latency t_s and reciprocal of the bandwidth t_c for the SMA Hydra Cluster. This can be done by repeatedly sending fixed-size messages from one process to another. When the receiving process receives the message, it should reply to the sending process. The ping-pong program developed in Question 1 can be used for this question. This process should then be repeated with messages of different sizes. After collecting the timings, construct a least-squares-fit line to resulting (message size,

time) pair. The intercept on the time vs message size curve should give an estimate of the startup cost t_s while the gradient of the slope will give an estimate for t_c .

3. Exercises based on sample MPI codes

(a) Compile and run the parallel integration code **integral.c** using different number of processes. Use the **Jumpshot** facility in MPICH to display the space-time status of your parallel computation (Show only one jumpshot graph for the case with the largest number of processes). What happens if you try to run it with just one processor? Run the sequential code **ori_integral.c** on one processor. What are the differences between running a sequential code and a parallelized code using one processor?

(b) A more accurate alternative to trapezoidal rule is Simpson's rule. It takes the following form in the interval of integration $[a, b]$ which is divided into n even intervals:

$$\int_a^b f(x)dx = \frac{h}{3} [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)]$$

Assume that you integrate using even number of processes p so that the ratio n/p is even. Modify the programs **ori_integral.c** and **integral.c** to estimate the value of the integral. State clearly how you have implemented message passing functions and compare the results with those from the trapezoidal rule (Show only one jumpshot graph for the case with the largest number of processes).

4. Parallelize your sequential code for calculating the value of **pi** by using Monte Carlo Method from Computational Lab 1:

This question will focus on the parallelization of the sequential code you have written in Computational Lab 1 to evaluate the value of **pi** by using Monte Carlo Method.

Write your own parallel codes by using (i) only MPI_SEND and MPI_RECV (Point-to-Point operations), and (ii) Collective operations. Choose the fixed total number of "darts" thrown at the board if you use Monte Carlo Method. Then run the code by varying the number of processor 2, 4, 8, 16 and 32. Record the communication time, computation time and the value your **pi**. Use the **Jumpshot** facility in MPICH to display the space-time status of your parallel computation. (Show one jumpshot graph only)

- Compare the value and precision of **pi** obtained from both parallel codes with that obtained from the serial code. (PI25DT = 3.141592653589793238462643 is the true **pi** value up to 25 digits of accuracy)
- Calculate the speedups and efficiencies for your own parallel code. Plot the curves of (α) Predicted and actual Speedup, (β) Predicted and actual Efficiency for your parallel version. Comment on your observations.
- Discuss your observations and make statements about the ratio of computation time to communication time. Explain clearly how you have estimated communication and computation time for this problem. That means show clearly where you insert MPI_Wtime() function to measure the running time.